## Introduction

The aim of this project is to design and implement a RESTful web application using Django and the Django REST Framework. The application has a relational database and highlights a few REST API endpoints that allow data to be retrieved, created, updated, and deleted using standard HTTP methods like GET, POST, PUT, and DELETE.

The application is built using a real world dataset from Kaggle which is the Airbnb listings in New York City. Django models were used to map the dataset into a structured relational format, with migrations to the database schema. This allows each listing to be stored as a structured record that can be queried properly. SQLite3 was used as the database engine, and the Django REST Framework was used to handle serialisation, request validation, and API view logic, simplifying the development of the REST endpoints while still controlling query and response behaviour.

The application includes filtered query endpoints that allow users to explore the dataset in more meaningful ways, like retrieving listings by borough or within a specified price range. A custom data loading script was implemented to import the dataset from a CSV file into the database. To ensure the application behaves correctly and reliably, unit tests were written for all main API endpoints.

## About Dataset & Justification

The dataset used was the New York City Airbnb Open Data dataset from public source : kaggle[1], which contains information about Airbnb listings across different neighbourhood groups in New York City. Each record in the dataset represents an individual listing and includes attributes such as neighbourhood group, room type, price, availability, and number of reviews and more. Overall the dataset is suitable for building and testing a RESTful web service.

For this project, the original dataset was cleaned and reduced to 7,000 rows. During the data cleaning process, rows with missing or invalid values, spaces and symbols in key fields were removed to ensure consistency and reliability of the stored data. The cleaned dataset was stored in CSV format and then imported into the application's SQLite database using a python data loading file.



```
[7]:
import pandas as pd
import re

# load the full Airbnb dataset
df = pd.read_csv("AB_NYC_2019.csv")

# drop all rows with any missing values
df_cleaned = df.dropna()

# clean the 'name' column
df_cleaned["name"] = df_cleaned["name"].str.strip()  # remove extra spaces
df_cleaned["name"] = df_cleaned["name"].apply(lambda x: re.sub(r'[^\w\s,.-]', '', x))  # remove symbols

# truncate long names (to fit CharField max_length in Django)
df_cleaned["name"] = df_cleaned["name"].str.slice(0, 200)

# random sample of 7000 rows
df_sampled = df_cleaned.sample(n=7000, random_state=42)
# remove Airbnblisting ID so Django can auto generate ids
df_sampled = df_sampled.drop(columns=["id", "host_id","latitude","longitude","last_review","reviews_per_month","calculated_host_listings_count"])
df_sampled = df_sampled.reset_index(drop=True)

# save the cleaned sample to a new CSV file
df_sampled.to_csv("airbnbtest_cleaned.csv", index=False)

# preview result
print(df_sampled.head())
```

This dataset was chosen because it is well structured and it suited the relational storage and querying and contains numerical attributes like price, availability, and review counts allowing filtering queries, while categorical attributes like neighbourhood (borough), neighbourhood, and room type enable more complex REST endpoints. For instance, listings can be filtered by location or price range.

Overall, the dataset is realistic  for demonstrating RESTful API design as it  supports the implementation of CRUD operations using POST PUT DELETE and GET, filtered queries while remaining small enough to load and process efficiently within a local development environment.

**Data Model & database design**

The application uses a Django model, AirbnbListing, to represent the Airbnb dataset in a relational database. Each instance of this model maps to a single Airbnb entry and stores the key attributes required for the REST API functionality. The model design closely reflects the structure of the cleaned CSV dataset while also ensuring that correct data types are enforced at the database level.

Attributes such as neighbourhood group, neighbourhood, and room type are represented using CharField, with suitable maximum lengths to prevent excessively large values from being stored. Numerical attributes, including price, minimum nights, number of reviews, and annual availability, are represented using IntegerField. This makes sure that numeric comparisons, filtering can be done efficiently and properly.

Core structure of the model :

api > models.py

```python
# import models module to define the database tables
from django.db import models
# model showing how one AirbnbListing showuld be
class AirbnbListing(models.Model):
    # store the fields according to how it should be stored in the databse
    name = models.CharField(max_length=255)
    host_name = models.CharField(max_length=100)
    neighbourhood_group = models.CharField(max_length=50)
    neighbourhood = models.CharField(max_length=100)
    room_type = models.CharField(max_length=50)
    price = models.IntegerField()
    minimum_nights = models.IntegerField()
    number_of_reviews = models.IntegerField()
    availability_365 = models.IntegerField()

    class Meta:
        # model name
        verbose_name_plural = "Airbnb"
    # show useful information like name of the airbnb and the bourough
    def __str__(self):
        return f"{self.name} ({self.neighbourhood_group})"

```

Django automatically provides a unique primary key (id) for each entry, which is used internally by the application for operations like record identification and updating it. This avoids depending on dataset identifiers and it simplifies REST endpoint design, especially for the update and delete operations. The database schema is managed using Django migrations, allowing changes to the model to be applied consistently.

The database used which is SQLite3 is suitable for local development and for medium sized datasets like the Airbnb data as it provides sufficient performance while maintaining simplicity and ease of setup. Overall, the data model is intentionally minimal but structured, supporting efficient querying, clean REST API design, and straightforward data loading.

**Architecture & data flow**

The application follows Django Model View Template architecture and uses Django REST Framework to provide REST API functionality. This separates data storage, request handling, and response generation, making the application easier to maintain and use.

The AirbnbListing model tells how listing data is stored in the SQLite database. Data is imported from a cleaned CSV file using a custom Django management command that reads the file with Pandas and creates database records automatically. This makes sure that the database can be populated consistently and efficiently.

Once stored, data is accessed through Django REST Framework views. Generic class based views are used for CRUD operations, while API views handle filtered queries like geting listings by borough or by price. These views interact with the database through Django's ORM and return JSON responses via serializers.

URL routing is managed through a urls.py file, which maps the incoming requests to the correct views. Serializers act as a middle layer, converting model instances into JSON and validating incoming POST and PUT requests before data is saved.

Apart from REST API, the application has a simple HTML interface for browsing listings and a Django admin interface for managing records. Overall, the architecture supports a clear data flow from CSV input to database storage and finally to the JSON responses.

**Endpoints**

Swagger was added to allow endpoints to be explored and tested from the browser.The application demonstrates six main RESTful API endpoints. These endpoints were designed to follow REST conventions, using standard HTTP methods and clear URL structures, with all responses returned in JSON format.



```
# urlpatterns map the url paths to the correct views
urlpatterns = [
    # endpoint for listing all the airbnb or creating a new one
    path('listings/', AirbnbListingList.as_view(), name='listings-list'),
    # endpoint for updating retirving and deleting by listing id
    path('listings/<int:id>/', AirbnbListingDetail.as_view(), name='listing-detail'),
    # endpoint to filter listings by borough
    path('listings/borough/<str:borough>/', ListingsByBorough.as_view(), name='listings-borough'),
    # endpoint to filter listing by under a particular price
    path('listings/under-price/<int:price>/', ListingsUnderPrice.as_view(), name='listings-under-price'),
```

The GET /api/listings/ endpoint returns a list of all Airbnb entires stored in the database. Pagination is used to ensure acceptable performance and usability when working with larger result sets. The same endpoint also supports POST /api/listings/, allowing new listings to be created by submitting validated JSON data through the API.

The PUT /api/listings/<id>/ endpoint allows an existing listing to be fully updated, while the DELETE /api/listings/<id>/ endpoint removes a listing from the database. Both endpoints use Django's automatically generated primary key (id) to uniquely identify records, making the operations easier.

Two filtered query endpoints were implemented to demonstrate more complex data access. The GET /api/listings/borough/<borough>/ endpoint retrieves all listings within a specified borough. The GET /api/listings/under-price/<price>/ endpoint returns all listings with a price less than or equal to the given value, enabling price based filtering.

- Swagger API Documentation

**REST API Endpoints**

- **GET** http://127.0.0.1:8000/api/listings/ - return a paginated list of all Airbnb listings
  http://127.0.0.1:8000/api/listings/
- **POST** http://127.0.0.1:8000/api/listings/ - create a new Airbnb listing
  http://127.0.0.1:8000/api/listings/
- **PUT** http://127.0.0.1:8000/api/listings/<id>/ - fully update an existing listing
  http://127.0.0.1:8000/api/listings/1/
- **DELETE** http://127.0.0.1:8000/api/listings/<id>/ - delete an existing listing
  http://127.0.0.1:8000/api/listings/1/
- **GET** http://127.0.0.1:8000/api/listings/borough/<borough>/ - filter listings by borough
  Example: http://127.0.0.1:8000/api/listings/borough/Manhattan/
- **GET** http://127.0.0.1:8000/api/listings/under-price/<price>/ - filter listings priced under a particular value
  Example: http://127.0.0.1:8000/api/listings/under-price/100/

## Data Loading

To populate the database, a data loading file was implemented along with importing a Django Basecommand. This helps the Airbnb dataset to be imported from a CSV file into the SQLite database automatically, without manually entering in the data.

```python
# basecommand to create custom django management command
from django.core.management.base import BaseCommand
# read and work with the csv
import pandas as pd
# import the AirbnbListing model to insert the data into the databse
from api.models import AirbnbListing

class Command(BaseCommand):
    # description will be shown to tell user
    help = 'Load the Airbnb CSV data into the database'

    def handle(self, *args, **options):
        # read clean csv
        df = pd.read_csv('data/airbnbtest_cleaned.csv')
        # go through each row in the dataframe
        for _, row in df.iterrows():

            # create airbnb listing object for every row
            # each field is mapped to the column in the csv
            AirbnbListing.objects.create(
                name=row['name'],
                host_name=row['host_name'],
                neighbourhood_group=row['neighbourhood_group'],
                neighbourhood=row['neighbourhood'],
                room_type=row['room_type'],
                price=int(row['price']),
                minimum_nights=int(row['minimum_nights']),
                number_of_reviews=int(row['number_of_reviews']),
                availability_365=int(row['availability_365']),
            )

        self.stdout.write(
            self.style.SUCCESS('CSV data loaded successfully!')
        )
```

Every row in the CSV is extracted and passed into the AirbnbListing model constructor. Django's ORM then creates a new AirbnbListing object and then stores it as a row in the SQLite database. This is done until all rows in the CSV file have been processed. Type conversions are also used to ensure that numerical values such as price are stored correctly. By using a management command, the data loading process can be triggered with a single command and easily run if the database needs to be reset.

This method makes sure that all relavent data is loaded consistently and ensures quick setup of the application. Overall, the loading approach improves reproducibility and aligns with best practices for managing structured datasets in Django applications

## Unit Testing

Unit testing was done to see and check how correct and reliable the REST API endpoints were being developed. APITestCase was used to create the tests which interact directly with the API, making sure that every endpoint behaves as expected under the conditions. When the tests are executed, Django automatically creates a temporary test database, preventing the main application data from being affected.

Seven unit tests were written, based on the six main REST API endpoints implemented in the application. These tests cover retrieving all listings, creating a new listing using POST, updating an existing listing using PUT, deleting a listing, and testing the two filtered query endpoints for borough based and under a particular price based retrieval. Each test checks the returned HTTP status codes and validates key aspects of the response data.

The unit tests were executed and all of the tests ran and completed successfully. This shows that the application's API endpoints function correctly and consistently, and that the changes to the code can be validated quickly through testing. Overall, including unit tests improves the application's behaviour and aligns with best practices for RESTful web service development.

```
(my_env) C:\Users\vinay\Downloads\Uni projects\Y3S1\Advanced Web Development\midterm_awd>python manage.py test
Found 7 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
..C:\Users\vinay\Downloads\Uni projects\Y3S1\Advanced Web Development\midterm_awd\my_env\Lib\site-packages\rest_framework\pagination.py:20
7: UnorderedObjectListWarning: Pagination may yield inconsistent results with an unordered object_list: <class 'api.models.AirbnbListing'>
 QuerySet.
  paginator = self.django_paginator_class(queryset, page_size)
.....
----------------------------------------------------------------------
Ran 7 tests in 0.021s

OK
Destroying test database for alias 'default'...
```

## Evaluation

The project provided insights into designing and implementing a RESTful web service using Django and Django REST Framework. One strength of the application is the modular structure, where the models, serializers, views, and URL configurations are properly separated. This structure made it easier to debug issues that came up during development, like performance problems when retrieving large datasets and some errors related to request handling for different HTTP methods.

A particularly important design decision was the use of pagination for the main listings endpoint. Initially, returning all listings in a single response caused the browser to become unresponsive on the local development machine making it crash. Hence pagination significantly improved performance and usability, showing the importance of considering the

response size and client limitations even when working with relatively small datasets. This change reflects a realistic challenge faced in API development and demonstrates the need to adapt implementation choices based on observed behaviour rather than theoretical assumptions.

Django REST Framework's generic class based views proved effective for implementing the CRUD operations with minimal templated code. However, implementing filtered endpoints like retrieving listings by borough or under a specific price required a deeper understanding of query sets and the URL parameter handling. These endpoints helped demonstrate more complex querying logic and reinforced the importance of careful URL design and query when working with real world data.

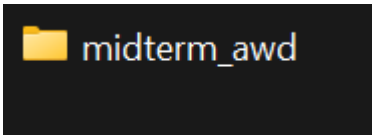
However the project also has its limitations. The application relies on SQLite3, which was suitable for development and met the assignment requirements, but would not be appropriate for a production scale system with concurrent users. Even if the Django admin interface provides data management capabilities, the public REST API does not currently implement very proper authentication or access control. In real deployment, unrestricted access to create, update, or delete records would be unsuitable.

Overall, the project meets the objectives and reflects the techniques taught in the module, while also taking note of practical considerations like the performance, scalability, and security that will come about in real world web development. These insights show how building a complete application highlights challenges that may not always be obvious in isolated examples and provide a strong foundation for future work in advanced web development.

**Running Instructions**

The required Python packages can then be installed using the following command:

The main folder is midterm_awd




cd midterm_awd_final_submission
cd midterm_awd_final
cd midterm_awd

- .\my_env\scripts\activate
- pip install -r requirements.txt

Database Setup and Data Loading

Once dependencies are installed, database migrations must be applied to create the required tables:

- python manage.py makemigrations

- python manage.py migrate

Load the airbnb dataset into the database using :

- python manage.py load_data

Running the Application

- python manage.py runserver

Once running, the main application page is accessible at: http://127.0.0.1:8000/ This page provides hyperlinks to all implemented REST API endpoints and admin page, as required by the assignment.

- Username: admin123
- Password: admin123

The admin interface allows staff users to view, add, update, and delete Airbnb listing records directly from the database.

Running Unit Tests Unit tests

- python manage.py test

This command creates a temporary test case database, runs all the tests, and reports the results in the console. All implemented tests pass successfully, confirming the correct behaviour of the API endpoints.

~ 1995 words